

Skill Week-11

1.To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components

pranay@LAPTOP-OP7R42IK

src/parser.h *

```
#ifndef PARSER_H
#define PARSER_H

int parse_command(char *input);

#endif
```

pranay@LAPTOP-OP7R42IK

src/parser.c *

```
#include <stdio.h>
#include <string.h>
#include "parser.h"

int parse_command(char *input) {
    if (input == NULL || strlen(input) == 0) {
        printf("Parser: Empty command.\n");
        return 0;
    }

    printf("Parser: Command accepted -> %s\n", input);

    return 1;
}
```

pranay@LAPTOP-OP7R42IK

src/executor.h *

```
#ifndef EXECUTOR_H
#define EXECUTOR_H

int execute_command(char *command);

#endif
```

pranay@LAPTOP-OP7R42IK

src/executor.c *

```
#include <stdio.h>
#include <stdlib.h>
#include "executor.h"

int execute_command(char *command) {
    int result;

    printf("Executor: Running command...\n");

    result = system(command);

    if (result != 0) {
        printf("Executor: Command failed.\n");
        return 0;
    }

    printf("Executor: Command completed successfully.\n");

    return 1;
}
```

pranay@LAPTOP-OP7R42IK

src/main.c *

```
#include <stdio.h>
#include <string.h>

#include "parser.h"
#include "executor.h"

int main() {
    char command[100];

    printf("Enter command: ");
    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    if (!parse_command(command)) {
        printf("Main: Invalid command.\n");
        return 1;
    }

    if (!execute_command(command)) {
        printf("Main: Execution failed.\n");
        return 1;
    }

    printf("Main: End-to-end execution successful.\n");

    return 0;
}
```

Output:

pranay@LAPTOP-OP7R42IK

```
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ ls
bin  src
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ nano src/parser.h
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ nano src/parser.c
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ nano src/executor.h
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ nano src/executor.c
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ nano src/main.c
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ gcc src/main.c src/parser.c src/executor.c -o bin/modular_shell
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ ./bin/modular_shell
Enter command: ls
Parser: Command accepted -> ls
Executor: Running command...
bin  src
Executor: Command completed successfully.
Main: End-to-end execution successful.
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ ./bin/modular_shell
Enter command: invalid_command_xyz
Parser: Command accepted -> invalid_command_xyz
Executor: Running command...
sh: 1: invalid_command_xyz: not found
Executor: Command failed.
Main: Execution failed.
```

2. To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

pranay@LAPTOP-OP7R42IK

src/error_handler.h *

```
#ifndef ERROR_HANDLER_H
#define ERROR_HANDLER_H

void log_error(const char *message);

#endif
```

pranay@LAPTOP-OP7R42IK

src/error_handler.c *

```
#include <stdio.h>
#include "error_handler.h"

void log_error(const char *message) {
    FILE *file = fopen("error.log", "a");

    if (file == NULL) {
        printf("Unable to open error log.\n");
        return;
    }

    fprintf(file, "ERROR: %s\n", message);

    fclose(file);

    printf("ERROR: %s\n", message);
}
```

pranay@LAPTOP-OP7R42IK

src/error_test.c *

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#include "error_handler.h"

int main() {
    char command[100];

    printf("Enter command: ");
    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    if (strlen(command) == 0) {
        log_error("Empty command entered.");
        printf("Recovering: Please enter a command next time.\n");
        return 1;
    }

    if (strstr(command, "&&&") != NULL) {
        log_error("Invalid command syntax.");
        printf("Recovering: Invalid syntax ignored.\n");
        return 1;
    }

    printf("Executing command...\n");

    int result = system(command);

    if (result != 0) {
        log_error("Command execution failed.");
        printf("Recovering: Shell remains active.\n");
        return 1;
    }

    printf("Command executed successfully.\n");

    return 0;
}
```

Output:

pranay@LAPTOP-OP7R42IK

```
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ nano src/error_handler.h
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ nano src/error_handler.c
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ nano src/error_test.c
```

pranay@LAPTOP-OP7R42IK

```
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ gcc src/error_test.c src/error_handler.c -o bin/error_test
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ ./bin/error_test
Enter command: pwd
Executing command...
/home/pranay/OS_Skill/Skill-Week-11
Command executed successfully.
```

3.To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

pranay@LAPTOP-OP7R42IK

src/memory_test.c *

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr;

    arr = malloc(5 * sizeof(int));

    if (arr == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
        arr[i] = i + 1;

    printf("Array values: ");

    for (int i = 0; i < 5; i++)
        printf("%d ", arr[i]);

    printf("\n");

    free(arr);
    arr = NULL;

    printf("Memory released successfully.\n");

    return 0;
}
```

Output:

pranay@LAPTOP-OP7R42IK

```
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ nano src/memory_test.c
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ gcc src/memory_test.c -o bin/memory_test -g
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ ./bin/memory_test
Array values: 1 2 3 4 5
Memory released successfully.
```

4.To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports

pranay@LAPTOP-OP7R42IK

src/test_memory.sh *

```
#!/bin/bash

echo "==== Memory Test Report ====="

echo "Test 1: Program execution"

output=$(./bin/memory_test)

echo "$output"

if echo "$output" | grep -q "Memory released successfully."; then
    echo "Test 1: PASS"
else
    echo "Test 1: FAIL"
fi

echo
echo "Test 2: Array values"

if echo "$output" | grep -q "1 2 3 4 5"; then
    echo "Test 2: PASS"
else
    echo "Test 2: FAIL"
fi

echo
echo "==== Test Summary ====="
echo "Testing completed."
```

Output:

pranay@LAPTOP-OP7R42IK

```
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ nano src/test_memory.sh
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ chmod +x src/test_memory.sh
pranay@LAPTOP-OP7R42IK:~/OS_Skill/Skill-Week-11$ ./src/test_memory.sh
==== Memory Test Report =====
Test 1: Program execution
Array values: 1 2 3 4 5
Memory released successfully.
Test 1: PASS

Test 2: Array values
Test 2: PASS

==== Test Summary =====
Testing completed.
```